

有限元总体刚度矩阵组装与桁架求解程序设计报告

一、作业概述

本次作业基于《2.3 系统总体刚度方程》课程内容，自主编写 Python 有限元程序，实现一维杆、二维桁架单元的**对号矩阵 LM 生成、单元刚度计算、总体刚度矩阵直接组装、缩减法边界条件求解、单元应力后处理**完整流程。程序完成作业给定 2 组标准算例校核，验证刚度矩阵对称性、奇异性、结构受力平衡等有限元核心性质。

1.1 学习目标

1. 根据单元连接数组 IEN 自动构建对号矩阵 LM;
2. 采用直接叠加法组装全局总体刚度矩阵;
3. 使用缩减法处理位移边界条件，求解节点位移与支座约束反力;
4. 提取单元局部位移，计算单元长度、方向余弦、应力、轴力;
5. 分析总体刚度矩阵对称、奇异、稀疏、对角元非负特性;
6. 掌握有限元标准五步流程：前处理→单元分析→刚度组装→方程求解→后处理。

1.2 程序整体架构

程序采用模块化函数划分，功能解耦：

1. 工具函数：矩阵对称校验、矩阵奇异性判断、整体受力平衡校验;
2. 前处理模块：JSON 模型读取、LM 对号矩阵生成、总自由度计算;
3. 单元模块：1D 杆单元刚度/应力、2D 桁架单元刚度/方向余弦/应力计算;
4. 组装模块：依据 LM 矩阵累加单元刚度，生成全局刚度 K;
5. 求解模块：缩减法分块刚度矩阵，求解未知位移、支座反力;
6. 后处理模块：输出单元几何参数、应力、轴力;
7. 主函数：串联全部流程，内置一维、二维两组标准算例 JSON 模型。

二、有限元核心理论实现说明

2.1 对号矩阵 LM 生成原理

设：节点总数`nnp`，单节点自由度`ndof`，每个单元 2 个节点`nen=2`；

单元局部自由度总数 = `nen × ndof`；

输入 IEN 为 1 基节点编号，程序内部统一转换为 0 基；

对号矩阵`LM[局部自由度编号, 单元号] = 对应全局自由度编号`；

全局自由度映射规则：`全局自由度 = 节点号 × ndof + 方向序号`。

二维桁架单元局部位移向量： $d^e = [u_1, v_1, u_2, v_2]^T$ ，LM 矩阵自动匹配每个局部位移对应的全局位置，为刚度组装提供寻址索引。

2.2 总体刚度矩阵直接组装算法

组装核心公式：

$$K[LM[a, e], LM[b, e]] += K_e[a, b]$$

循环遍历全部单元，先计算当前单元刚度矩阵 K_e ；再双重循环遍历单元所有局部自由度，通过 LM 矩阵找到对应全局刚度位置，将单元刚度叠加至全局刚度矩阵。

组装天然保证总体刚度矩阵对称，程序内置对称校验函数验证。

2.3 缩减法边界条件处理

将全部自由度分为约束已知自由度 E、自由未知自由度 F，对刚度矩阵、载荷向量分

块：
$$\begin{bmatrix} K_{FF} & K_{FE} \\ K_{EF} & K_{EE} \end{bmatrix} \begin{bmatrix} d_F \\ d_E \end{bmatrix} = \begin{bmatrix} f_F \\ f_E \end{bmatrix}$$

已知约束位移 d_E ，移项求解自由位移：

$$K_{FF} \mathbf{d}_F = \mathbf{f}_F - K_{FE} \mathbf{d}_E$$

求解 d_F 后拼接完整位移向量 d ；

支座约束反力计算公式：

$$\mathbf{R} = K_{EF} \mathbf{d}_F + K_{EE} \mathbf{d}_E - \mathbf{f}_E$$

物理意义：支座对结构施加的约束力，整体结构满足

$$\mathbf{f}_{\text{外载荷}} + \mathbf{R} = \mathbf{0}$$

2.4 桁架单元应力计算

1. 一维杆单元

单元长度 L ，弹性模量 E ，局部位移 $d^e = [u_1, u_2]$

$$\sigma = \frac{E}{L}(-u_1 + u_2), \quad N = \sigma A$$

2. 二维桁架单元

单元方向余弦 $c = \Delta x/L$ ， $s = \Delta y/L$ 局部位移 $d^e = [u_1, v_1, u_2, v_2]$

$$\sigma = \frac{E}{L}(-cu_1 - sv_1 + cu_2 + sv_2), \quad N = \sigma A$$

规定：轴力 $N > 0$ 受拉， $N < 0$ 受压。

三、算例 2：二维两杆桁架完整计算结果与校核

3.1 模型输入 JSON 参数

```
{  
  "Title": "2D two truss elements",  
  "nsd": 2,  
  "ndof": 2,  
  "nnp": 3,  
  "nel": 2,  
  "nen": 2,  
  "E": [1.0, 1.0],  
  "CArea": [1.0, 1.0],  
  "x": [1.0, 0.0, 1.0],  
  "y": [0.0, 0.0, 1.0],  
  "IEN": [[1, 3], [2, 3]],  
  "fixed_dof": [1, 2, 3, 4],  
  "fixed_value": [0.0, 0.0, 0.0, 0.0],  
  "force_dof": [5, 6],  
  "force_value": [10.0, 0.0]
```

}

模型说明:

节点 1 坐标(1,0)、节点 2(0,0)、节点 3(1,1);

单元 1: 1-3 (竖直杆), 单元 2: 2-3 (斜杆);

节点 1、2 完全固定, 节点 3 施加水平向右外力 $F_x=10$;

材料 $E_1=E_2=1$, 截面积 $A_1=A_2=1$ 。

3.2 程序输出关键结果

(1) 对号矩阵 LM

[[0 2]

[1 3]

[4 4]

[5 5]]

单元 1 局部自由度对应全局: 0(u_1),1(v_1),4(u_3),5(v_3)

单元 2 局部自由度对应全局: 2(u_2),3(v_2),4(u_3),5(v_3)

LM 生成逻辑完全符合节点自由度映射规则。

(2) 总体刚度矩阵 K (保留 6 位小数)

$$K = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0.353553 & 0.353553 & -0.353553 & -0.353553 \\ 0 & 0 & 0.353553 & 0.353553 & -0.353553 & -0.353553 \\ 0 & 0 & -0.353553 & -0.353553 & 0.353553 & 0.353553 \\ 0 & -1 & -0.353553 & -0.353553 & 0.353553 & 0.353553 \end{bmatrix}$$

1. 对称校验: 矩阵最大差值 0.00×10^0 , 严格对称;

2. 无约束整体行列式 $\det(K) = 0.00 \times 10^0$, 矩阵奇异;

物理意义：无边界约束桁架存在刚体平动、转动，刚度矩阵奇异，无法直接求解；

3. 施加约束后缩减刚度 K_{eff} 行列式 0.354，非奇异，方程可唯一求解。

(3) 整体受力平衡校验

- 外载荷总分量： $F_x = 10, F_y = 0$

- 支座反力总分量： $R_x = 10, R_y = 0$

合力残差 $f + R = [0,0]$ ，结构满足静力学平衡。

(4) 节点位移结果

节点 1 $u=0.0000000000, v=0.0000000000$

节点 2 $u=0.0000000000, v=0.0000000000$

节点 3 $u=38.2842712475, v=-10.0000000000$

与理论标准解 $u_3 \approx 38.284271, v_3 = -10$ 完全吻合。

(5) 支座约束反力（标注节点+方向）

全局自由度 1 (节点 1, u) 支座反力 $R = 0.0000000000$

全局自由度 2 (节点 1, v) 支座反力 $R = 10.0000000000$

全局自由度 3 (节点 2, u) 支座反力 $R = -10.0000000000$

全局自由度 4 (节点 2, v) 支座反力 $R = -10.0000000000$

x 向总反力： $0-10=-10$ ，平衡水平外力 10； y 向总反力： $10-10=0$ ，无竖向载荷，受力平衡。

(6) 单元后处理结果

1.单元 1 (1-3 竖直杆)

长度 $L=1.0$, 方向余弦 $c=0, s=1$;

应力 $\sigma=-10.0$, 轴力 $N=-10.0$ (负号代表受压)。

2.单元 2 (2-3 斜杆)

长度 $L \approx 1.4142135624$, 方向余弦 $c = s \approx 0.7071067812$;

应力 $\sigma \approx 14.1421356237$, 轴力 $N \approx 14.1421356237$ (正号代表受拉)。

单元应力、轴力与作业给定理论解完全一致。

四、总体刚度矩阵性质分析

4.1 对称性

程序校验全局刚度矩阵 $K - K^T$ 最大差值为 0, 严格对称。

原理: 单元刚度矩阵 K_e 本身对称, 叠加组装后总体刚度保持对称; 物理意义: 刚度互等定理, $K_{ij} = K_{ji}$,

第 j 自由度单位位移在 i 自由度产生的力等于第 i 自由度单位位移在 j 自由度产生的力。

4.2 奇异性

无边界约束时 $\det(K)=0$, 矩阵奇异;

物理本质: 自由桁架无约束, 存在刚体位移 (平动、转动), 位移解不唯一;

施加足够位移约束后, 缩减刚度 K_{FF} 非奇异, 方程存在唯一位移解。

4.3 稀疏性

本次算例 6 阶刚度矩阵存在大量 0 元素, 例如第 1 行全部为 0;

原因: 节点 1 水平自由度无杆件提供水平刚度贡献, 对应行、列全部为 0;

工程大型结构中刚度矩阵极度稀疏, 可采用稀疏存储提升计算效率。

4.4 对角元非负性

刚度矩阵所有对角元素均大于等于 0;

物理意义：对角元 K_{ii} 代表仅第 i 自由度产生单位位移、其余自由度固定时，需要施加在 i 自由度的外力，外力与位移同向，恒为正值。

4.5 刚度矩阵第 j 列物理意义

固定所有自由度，仅令第 j 全局自由度产生单位位移 $d_j = 1$ ，其余位移为 0；此时全局载荷向量 f 即为总体刚度矩阵第 j 列。

数值含义：为实现 j 方向单位位移，所有全局自由度上需要施加的外力大小。

五、一维算例简要说明（备用校核）

算例 1 为 3 节点一维两杆轴向拉伸结构，程序可切换运行；

1. 组装得到总体刚度矩阵与作业标准矩阵完全匹配；

$$K = \begin{bmatrix} 100 & -100 & 0 \\ -100 & 300 & -200 \\ 0 & -200 & 200 \end{bmatrix}$$

2. 节点位移 $d_1=0, d_2=0.1, d_3=0.15$;

3. 节点 1 支座反力为 -10，与外载荷 10 平衡；

4. 无约束 K 奇异，约束后缩减矩阵非奇异，全部校核通过。

六、总结

1. 程序完整实现有限元桁架求解五大核心模块，严格遵循课程总体刚度矩阵组装理论；

2. LM 对号矩阵、单元刚度、直接组装、缩减法边界处理、应力后处理全部计算正确；

3. 两组标准算例位移、反力、单元应力均与理论解完全吻合，受力平衡校验通过；

4. 程序可清晰输出刚度矩阵对称、奇异、稀疏等全部要求分析指标。

七、代码及结果展示

```
import json
import numpy as np

# ===== 工具辅助函数（全部重构优化） =====
def check_symmetric(K, tol=1e-10):
    """校验刚度矩阵对称性"""
    diff = np.max(np.abs(K - K.T))
    return diff < tol, diff

def is_singular(mat, tol=1e-10):
    """稳定判奇异：用秩判断，替代易溢出的行列式"""
    if mat.shape[0] != mat.shape[1]:
        return True, -1.0
    rank = np.linalg.matrix_rank(mat, tol=tol)
    singular = rank < mat.shape[0]
    det = np.linalg.det(mat) if mat.shape[0] <= 10 else np.nan
    return singular, det

def check_force_balance(f_global, R, fixed_dofs, free_dofs, ndof, tol=1e-8):
    """分方向平衡校验（1D/2D 通用）"""
    total_dof = len(f_global)
    f_total = np.zeros(ndof)
    r_total = np.zeros(ndof)
    # 累加外载荷
    for dof in free_dofs:
        axis = dof % ndof
        f_total[axis] += f_global[dof]
    # 累加支座反力
    for idx, dof in enumerate(fixed_dofs):
        axis = dof % ndof
        r_total[axis] += R[idx]
    # 合力  $f + R \approx 0$ 
    residual = f_total + r_total
    balance_ok = np.all(np.abs(residual) < tol)
    return residual, balance_ok, f_total, r_total
```

```
# ===== 1. 前处理模块 =====
```

```
def load_model(json_str=None, json_path=None):  
    if json_path is not None:  
        with open(json_path, "r", encoding="utf-8") as f:  
            data = json.load(f)  
    else:  
        data = json.loads(json_str)  
    return data
```

```
def build_LM(model):  
    nnp = model["nnp"]  
    ndof = model["ndof"]  
    nel = model["nel"]  
    nen = model["nen"]  
    IEN = np.array(model["IEN"], dtype=int) - 1  
    n_local_dof = nen * ndof  
    LM = np.zeros((n_local_dof, nel), dtype=int)  
    for e in range(nel):  
        nodes = IEN[e]  
        local_d = 0  
        for node in nodes:  
            for d in range(ndof):  
                LM[local_d, e] = node * ndof + d  
                local_d += 1  
    return LM
```

```
def get_total_dof(model):  
    return model["nnp"] * model["ndof"]
```

```
# ===== 2. 单元模块（增加浮点保护） =====
```

```
def element_1d_bar(x1, x2, E, A):  
    dx = x2 - x1  
    L = abs(dx)  
    if L < 1e-12:  
        raise ValueError(f"单元长度过小 L={L:.2e}, 单元退化")  
    ke_coeff = E * A / L  
    Ke = ke_coeff * np.array([[1, -1], [-1, 1]])
```

```
return Ke, L
```

```
def stress_1d(E, A, L, de):  
    sigma = E / L * (-de[0] + de[1])  
    N = sigma * A  
    return sigma, N
```

```
def element_2d_truss(x1, y1, x2, y2, E, A):  
    dx = x2 - x1  
    dy = y2 - y1  
    L = np.hypot(dx, dy)  
    if L < 1e-12:  
        raise ValueError(f"单元长度过小 L={L:.2e}, 单元退化")  
    c = dx / L  
    s = dy / L  
    coeff = E * A / L  
    Ke = coeff * np.array([  
        [c**2, c*s, -c**2, -c*s],  
        [c*s, s**2, -c*s, -s**2],  
        [-c**2, -c*s, c**2, c*s],  
        [-c*s, -s**2, c*s, s**2]  
    ])  
    return Ke, L, c, s
```

```
def stress_2d(E, A, L, c, s, de):  
    sigma = E / L * (-c*de[0] - s*de[1] + c*de[2] + s*de[3])  
    N = sigma * A  
    return sigma, N
```

```
# ===== 3. 总体刚度组装  
=====
```

```
def assemble_global_K(model, LM):  
    ndof_tot = get_total_dof(model)  
    K = np.zeros((ndof_tot, ndof_tot), dtype=np.float64)  
    x = np.array(model["x"])  
    y = np.array(model["y"])  
    IEN = np.array(model["IEN"], dtype=int) - 1  
    E_list = model["E"]
```

```

A_list = model["CArea"]
nel = model["nel"]
ndof = model["ndof"]

for e in range(nel):
    n1, n2 = IEN[e]
    x1, y1 = x[n1], y[n1]
    x2, y2 = x[n2], y[n2]
    E = E_list[e]
    A = A_list[e]
    if ndof == 1:
        Ke, _ = element_1d_bar(x1, x2, E, A)
    elif ndof == 2:
        Ke, _, _, _ = element_2d_truss(x1, y1, x2, y2, E, A)
    else:
        raise NotImplementedError("仅支持 1D 杆 / 2D 桁架")
    n_local = Ke.shape[0]
    for a in range(n_local):
        for b in range(n_local):
            gi = LM[a, e]
            gj = LM[b, e]
            K[gi, gj] += Ke[a, b]

return K

```

===== 4. 缩减法求解（增加异常捕获）
=====

```

def solve_reduction(K, f_vec, fixed_dof_1, fixed_vals):
    ndof_tot = len(f_vec)
    fixed = np.array(fixed_dof_1, int) - 1
    free = np.array([i for i in range(ndof_tot) if i not in fixed])
    nf = len(free)

    Kff = K[np.ix_(free, free)]
    Kfe = K[np.ix_(free, fixed)]
    Kef = K[np.ix_(fixed, free)]
    Kee = K[np.ix_(fixed, fixed)]

    df = np.array(fixed_vals, float)

```

```

ff = f_vec[free]
rhs = ff - Kfe @ df

# 捕获缩减矩阵奇异报错
try:
    d_free = np.linalg.solve(Kff, rhs)
except np.linalg.LinAlgError:
    raise RuntimeError("缩减刚度矩阵 Kff 奇异，边界约束不足，结构几何可变!")
")

d_total = np.zeros(ndof_tot)
d_total[free] = d_free
d_total[fixed] = df

# 支座反力
R = Kef @ d_free + Kee @ df - f_vec[fixed]
return d_total, R, free, fixed, Kff

# ===== 5. 后处理 =====
def post_process(model, LM, d_total):
    elem_results = []
    print("\n" + "="*60 + " 单元后处理结果 " + "="*60)
    x = np.array(model["x"])
    y = np.array(model["y"])
    IEN = np.array(model["IEN"], int) - 1
    E_list = model["E"]
    A_list = model["CArea"]
    nel = model["nel"]
    ndof = model["ndof"]

    for e in range(nel):
        n1, n2 = IEN[e]
        x1, y1 = x[n1], y[n1]
        x2, y2 = x[n2], y[n2]
        E = E_list[e]
        A = A_list[e]
        de_idx = LM[:, e]
        de = d_total[de_idx]

```

```

    res = {"elem_id": e+1}
    print(f"\n==== 单元 {e+1} =====")
    if ndof == 1:
        Ke, L = element_1d_bar(x1, x2, E, A)
        sigma, N = stress_1d(E, A, L, de)
        res.update({"L":L, "sigma":sigma, "N":N})
        print(f"单元长度 L = {L:.10f}")
        print(f"应力  $\sigma$  = {sigma:.10f}")
        print(f"轴力 N = {N:.10f} (拉力为正, 压力为负)")
    elif ndof == 2:
        Ke, L, c, s = element_2d_truss(x1, y1, x2, y2, E, A)
        sigma, N = stress_2d(E, A, L, c, s, de)
        res.update({"L":L, "c":c, "s":s, "sigma":sigma, "N":N})
        print(f"单元长度 L = {L:.10f}")
        print(f"方向余弦 c={c:.10f}, s={s:.10f}")
        print(f"应力  $\sigma$  = {sigma:.10f}")
        print(f"轴力 N = {N:.10f} (拉力为正, 压力为负)")
    elem_results.append(res)
return elem_results

# ===== 主求解流程 (增加理论解对比输出)
# =====
def run_fem(json_model_str):
    model = load_model(json_str=json_model_str)
    LM = build_LM(model)
    ndof_tot = get_total_dof(model)
    ndof = model["ndof"]
    nnp = model["nnp"]
    print("="*70 + " 对号矩阵 LM (局部自由度×单元) " + "="*70)
    print(LM)

    # 组装全局刚度
    K = assemble_global_K(model, LM)
    print("\n" + "="*70 + " 总体刚度矩阵 K (保留 6 位小数) " + "="*70)
    print(np.round(K, 6))

    # 对称校验
    sym_flag, diff = check_symmetric(K)

```

```

print(f"\n 【刚度矩阵对称校验】 最大误差 = {diff:.2e}, 对称: {sym_flag}")

# 无约束时奇异性校验
sing_K, detK = is_singular(K)
print(f"\n 【无边界约束整体 K】 行列式 det(K) = {detK:.2e}, 矩阵奇异: {sing_K}")

# 构建载荷向量
f = np.zeros(ndof_tot)
f_dof_1 = model["force_dof"]
f_val = model["force_value"]
f_idx = np.array(f_dof_1, int) - 1
f[f_idx] = f_val

# 缩减法求解
fix_dof_1 = model["fixed_dof"]
fix_val = model["fixed_value"]
d_total, R, free_dofs, fixed_dofs, Kff = solve_reduction(K, f, fix_dof_1, fix_val)

# 缩减刚度奇异性
sing_Kff, detKff = is_singular(Kff)
print(f"\n 【约束后缩减刚度 Kff】 行列式 det(Kff) = {detKff:.2e}, 奇异: {sing_Kff}")

# 分方向平衡校验
residual, balance_ok, f_sum, r_sum = check_force_balance(f, R, fixed_dofs, free_dofs,
ndof)
print(f"\n 【整体力平衡校验】 ")
print(f"外载荷分轴总和: {f_sum}")
print(f"支座反力分轴总和: {r_sum}")
print(f"合力残差 f+R = {residual}, 平衡满足: {balance_ok}")

# 节点位移输出
print("\n" + "="*70 + " 全部节点位移结果 " + "="*70)
for node in range(nnp):
    if ndof == 1:
        u = d_total[node*1]
        print(f"节点{node+1}  u = {u:.10f}")
    elif ndof == 2:
        u = d_total[node*2]

```

```

        v = d_total[node*2 + 1]
        print(f"节点{node+1}  u={u:.10f}, v={v:.10f}")

# 支座反力输出（增加节点、方向标识）
print("\n" + "=" * 70 + " 约束自由度支座反力 " + "=" * 70)
for idx, gdof in enumerate(fixed_dofs):
    node_id = gdof // ndof + 1
    axis = "u" if gdof % ndof == 0 else "v"
    print(f"全局自由度 {gdof + 1} (节点{node_id}, {axis}) 支座反力 R =
{R[idx]:.10f}")

# 单元应力轴力
elem_res = post_process(model, LM, d_total)

# 理论解对照输出（作业标准解）
print("\n" + "=" * 70 + " 作业标准理论解对照 " + "=" * 70)
if model["Title"] == "1D two bar elements":
    print("理论位移: d1=0.0, d2=0.1, d3=0.15")
    print("理论支座反力: 节点 1 反力 = -10.0")
    print("理论总体刚度矩阵: \n[[100,-100,0],[-100,300,-200],[0,-200,200]]")
elif model["Title"] == "2D two truss elements":
    print("理论节点 3 位移: u3≈38.284271247, v3≈-10.0")
    print("理论单元应力: 单元 1  $\sigma \approx -10.0$ , 单元 2  $\sigma \approx 14.1421356237$ ")
return K, d_total, R, elem_res, Kff

# ===== 算例 JSON 定义（严格匹配作业参数）
=====

# 算例 1: 一维两单元杆 严格满足 E1A1/L1=100, E2A2/L2=200
CASE1_1D = ""
{
    "Title": "1D two bar elements",
    "nsd":1,
    "ndof":1,
    "nnp":3,
    "nel":2,
    "nen":2,
    "E": [100, 200],
    "CArea": [1, 1],

```

```

"x": [0, 1, 2],
"y": [0, 0, 0],
"lEN": [[1,2],[2,3]],
"fixed_dof": [1],
"fixed_value": [0.0],
"force_dof": [3],
"force_value": [10.0]
}
'''

```

算例 2: 二维两杆桁架 完全匹配题目输入

```

CASE2_2D = '''
{
  "Title": "2D two truss elements",
  "nsd": 2,
  "ndof": 2,
  "nnp": 3,
  "nel": 2,
  "nen": 2,
  "E": [1.0, 1.0],
  "CArea": [1.0, 1.0],
  "x": [1.0, 0.0, 1.0],
  "y": [0.0, 0.0, 1.0],
  "lEN": [[1, 3], [2, 3]],
  "fixed_dof": [1, 2, 3, 4],
  "fixed_value": [0.0, 0.0, 0.0, 0.0],
  "force_dof": [5, 6],
  "force_value": [10.0, 0.0]
}
'''

```

===== 运行入口 =====

```

if __name__ == "__main__":

```

```

    # 切换算例: 注释/取消注释

```

```

    # model_json = CASE1_1D

```

```

    model_json = CASE2_2D

```

```

    K_global, disp, reaction, elem_data, Kff = run_fem(model_json)

```

```
# 单元轴力汇总
print("\n==== 全部单元轴力汇总 ====")
for elem in elem_data:
    print(f"单元{elem['elem_id']} 轴力 N = {elem['N']:.10f}")
```

结果：

C:\Users\Administrator\PyCharmMiscProject\venv\Scripts\python.exe
C:\Users\Administrator\PyCharmMiscProject\第五次作业.py

```
=====
===== 对号矩阵 LM (局部自由度×单元)
=====
=====
```

```
[[0 2]
 [1 3]
 [4 4]
 [5 5]]
```

```
=====
===== 总体刚度矩阵 K (保留 6 位小数)
=====
=====
```

```
[[ 0.      0.      0.      0.      0.      0.      ]
 [ 0.      1.      0.      0.      0.     -1.      ]
 [ 0.      0.      0.353553  0.353553 -0.353553 -0.353553]
 [ 0.      0.      0.353553  0.353553 -0.353553 -0.353553]
 [ 0.      0.     -0.353553 -0.353553  0.353553  0.353553]
 [ 0.     -1.     -0.353553 -0.353553  0.353553  1.353553]]
```

【刚度矩阵对称校验】 最大误差 = 0.00e+00, 对称: True

【无边界约束整体 K】行列式 $\det(K) = 0.00e+00$, 矩阵奇异: True

【约束后缩减刚度 Kff】行列式 $\det(K_{ff}) = 3.54e-01$, 奇异: False

【整体力平衡校验】

外载荷分轴总和: [10. 0.]

支座反力分轴总和: [-10. 0.]

合力残差 $f+R = [0. 0.]$, 平衡满足: True

=====
===== 全部节点位移结果
=====

节点 1 $u=0.0000000000, v=0.0000000000$

节点 2 $u=0.0000000000, v=0.0000000000$

节点 3 $u=38.2842712475, v=-10.0000000000$

=====
===== 约束自由度支座反力
=====

全局自由度 1 (节点 1, u) 支座反力 $R = 0.0000000000$

全局自由度 2 (节点 1, v) 支座反力 $R = 10.0000000000$

全局自由度 3 (节点 2, u) 支座反力 $R = -10.0000000000$

全局自由度 4 (节点 2, v) 支座反力 $R = -10.0000000000$

=====
===== 单元后处理结果
=====

====

==== 单元 1 ====

单元长度 $L = 1.0000000000$

方向余弦 $c=0.0000000000, s=1.0000000000$

应力 $\sigma = -10.0000000000$

轴力 $N = -10.0000000000$ (拉力为正, 压力为负)

==== 单元 2 ====

单元长度 $L = 1.4142135624$

方向余弦 $c=0.7071067812, s=0.7071067812$

应力 $\sigma = 14.1421356237$

轴力 $N = 14.1421356237$ (拉力为正, 压力为负)

=====
===== 作业标准理论解对照
=====

理论节点 3 位移: $u_3 \approx 38.284271247, v_3 \approx -10.0$

理论单元应力: 单元 1 $\sigma \approx -10.0$, 单元 2 $\sigma \approx 14.1421356237$

==== 全部单元轴力汇总 ====

单元 1 轴力 $N = -10.0000000000$

单元 2 轴力 $N = 14.1421356237$

进程已结束, 退出代码为 0